PilotSwarm Eval Harness: V1 Scope & Phase-Wise Plan

1. Overview

This document defines the v1 scope for the PilotSwarm Eval Harness and a phase-wise plan to build it out incrementally. It follows up on the full proposal shared earlier and incorporates feedback on starting simple with unit-level tool call evals before expanding.

V1 goal: given a specific input, can the LLM generate the tool calls we expect? Deterministic, code-graded assertions. No LLM judge, no human review, no Langfuse. Just a thin runner, golden datasets, and pass/fail results.

---

2. V1 Scope: Unit Test Layer

What "unit test" means here

Each eval task follows the pattern:

```
Input (prompt + context) → LLM turn → Assert (tool calls, params, ordering)
```

The assertions are deterministic code checks against CMS state and orchestration events. No subjective grading. A test either passes or fails.

Infrastructure

V1 reuses the existing test infrastructure rather than building new platform components:

Component	V1 Approach
Runner	Vitest-based, extends existing `withClient()` pattern
Datasets	JSON fixtures alongside test files
Grading	Code assertions via existing CMS helpers
Reporting	Console output + JSON results file
CI	Runs as part of existing `scripts/run-tests.sh`

The eval harness lives in `packages/sdk/test/eval/` (not a separate package yet). This keeps it close to the existing test helpers and avoids premature abstraction.

What V1 does NOT include

- LLM-as-judge grading
- Langfuse integration
- OTel tracing
- Chaos/fault injection
- Human-in-the-loop review
- Separate `packages/eval` package (that comes later when the harness stabilizes)

---

3. V1 Test Scenarios

3.1 Tool Call Correctness

Given a prompt that requires a specific tool, does the LLM call it with the correct parameters?

Scenario	Input	Expected
Single tool call	"Add 2 and 3"	`test_add({ a: 2, b: 3 })` called
Multi-param tool	"What's the weather in Tokyo in Celsius?"	`test_weather({ city: "Tokyo", unit: "celsius" })`
Tool selection	"Multiply 4 by 5" (both add and multiply available)	`test_multiply` called, not `test_add`
No tool needed	"What is 2+2?" (no tools registered)	LLM responds directly, no tool call

3.2 Multi-Tool Sequencing

When a task requires multiple tool calls, does the LLM sequence them correctly?

Scenario	Input	Expected
Sequential tools	"Add 2+3, then multiply the result by 4"	`test_add` then `test_multiply` in order
Independent tools	"Get weather for Tokyo and London"	Both `test_weather` calls made (order flexible)

3.3 Sub-Agent Spawn

Does the LLM correctly use `spawn_agent` to delegate work?

Scenario	Input	Expected
Basic spawn	"Create a sub-agent to research X"	`spawn_agent` tool called with task description
Named agent spawn	"Use the analyzer agent for this"	`spawn_agent` with correct `agentName`
Multiple spawns	"Create 3 agents to work on A, B, C"	3 `spawn_agent` calls with distinct tasks

3.4 Agent-to-Agent Messaging

Does the LLM correctly use messaging tools between parent and sub-agents?

Scenario	Input	Expected
Parent to sub	"Tell agent X to also check Y"	`message_agent` called with correct agent ID
Check agent status	"What are my agents doing?"	`check_agents` tool called

3.5 Interrupt Handling

Does the LLM handle user and agent interrupts correctly?

Scenario	Input	Expected
User interrupt mid-turn	Send cancel while agent is running	Session transitions to cancelled state
Cancel sub-agent	"Cancel agent X"	`cancel_agent` tool called with correct ID
Cancel all sub-agents	"Stop everything"	Cancel commands issued for all active sub-agents

3.6 Sub-Agent Cleanup

Does the LLM clean up sub-agents properly when work is done?

Scenario	Input	Expected
Completed sub-agents	Sub-agent finishes task	Parent acknowledges completion, no orphans
Failed sub-agent	Sub-agent hits error	Parent detects failure, handles gracefully

3.7 Session Stickiness

Does the LLM handle session resume correctly?

Scenario	Input	Expected
Resume conversation	Resume session, send follow-up	LLM has prior context, responds coherently
Resume after idle	Session idle for extended period, then resume	Session rehydrates, context intact

3.8 Quota/Token Failure Handling

Does the system handle Copilot session failures gracefully?

Scenario	Input	Expected
Token quota exceeded	Simulate quota error during turn	Session reports error, does not crash
Rate limit hit	Simulate rate limit response	Orchestration retries or reports cleanly

3.9 Checkpoint Recovery (Worker Crash)

How well does the LLM recover from the last checkpointed context after a worker crash?

Scenario	Input	Expected
Crash mid-turn	Kill worker during active turn, restart	Session resumes from last checkpoint
Recovery quality	Send follow-up after crash recovery	LLM response is coherent with pre-crash context
Data loss measurement	Compare pre-crash and post-crash CMS state	Quantify events/state lost in crash window

---

4. Test Parameter Matrix

Each test scenario runs across a configurable parameter matrix to surface how different settings affect behavior.

Parameters

Parameter	Values	What it tests
Model	Default model, plus 2-3 alternatives from the model catalog	Does tool call accuracy vary by model?
Context window	Small (4K tokens), medium (16K), large (128K+)	Do tool calls degrade with constrained context? Simulated by padding conversation history to fill the window before the eval prompt.
Reasoning strength	Standard, extended thinking (where supported)	Does deeper reasoning improve tool selection?
Compaction settings	Disabled, aggressive (low threshold), default	Does compaction affect tool call accuracy?

How the matrix works

The runner accepts a matrix config:

```
{
  "models": ["github-copilot:gpt-4o", "github-copilot:gpt-5.4"],
  "contextWindows": ["small", "medium", "large"],
  "compaction": ["disabled", "default", "aggressive"],
  "reasoning": ["standard", "extended"]
}
```

Each test scenario runs once per combination. Results are grouped by parameter so you can see: "tool call accuracy is 95% on gpt-4o but 88% on gpt-5.4 with aggressive compaction."

For v1, the default is single-model (the repo's default). The matrix is opt-in for targeted investigations.

---

5. Phase-Wise Implementation Plan

Phase 1: Runner + Core Unit Tests

Goal: Get the eval loop working end-to-end with the simplest possible scenarios.

Scope:

- Eval runner that executes test scenarios via `withClient()` and reports pass/fail
- JSON fixture format for test datasets
- 5-8 core tool call correctness tests (sections 3.1 and 3.2)
- Console + JSON output
- Runs via `scripts/run-tests.sh --suite=eval`

Success criteria: Run `./scripts/run-tests.sh --suite=eval` and get a green/red report for basic tool call assertions.

Phase 2: Expanded Scenarios + Parameter Matrix

Goal: Cover all v1 scenarios and enable model/config comparison.

Scope:

- Add sub-agent spawn tests (3.3)
- Add agent messaging tests (3.4)
- Add interrupt and cleanup tests (3.5, 3.6)
- Add session stickiness tests (3.7)
- Add quota/failure handling tests (3.8)
- Implement parameter matrix runner (model, context window, compaction, reasoning)
- Comparison output: results table by parameter combination

Success criteria: Full v1 scenario suite runs across 2+ models with clear comparison output.

Phase 3: Crash Recovery + Durability Evals

Goal: Test checkpoint recovery and measure data loss from worker crashes.

Scope:

- Worker crash simulation (kill and restart during active turn)
- Post-crash recovery quality measurement
- Data loss quantification (pre-crash vs post-crash CMS state)
- Checkpoint fidelity assertions
- Compaction recovery tests (crash during compaction)

Success criteria: Automated crash-and-recover tests that report data loss metrics and recovery quality scores.

Phase 4: Functional & Multi-Agent Evals

Goal: Move beyond unit tests into multi-turn, multi-agent evaluation.

Scope:

- Multi-turn conversation evals (does context carry across turns?)
- Sub-agent orchestration evals (spawn, delegate, collect results)
- Nested sub-agent trees (parent > child > grandchild)
- Agent-to-agent interrupt propagation
- Session policy enforcement
- System agent lifecycle (sweeper, resource manager auto-start)

Success criteria: End-to-end multi-agent workflows execute and produce scored results.

Phase 5: LLM Judge + Observability + CI Gates

Goal: Add subjective quality grading and integrate into CI/CD.

Scope:

- LLM-as-judge grading with rubrics (quality, coherence, instruction following)
- Langfuse integration for traces and prompt versioning
- Multi-trial statistical evaluation (Mann-Whitney U for regression detection)

- CI/CD gates: code-graded on PR, smoke eval on merge, full suite nightly
- Promote eval harness to `packages/eval` as a standalone package
- Human-in-the-loop review dashboard

Success criteria: Eval runs in CI, catches regressions automatically, and provides observability through Langfuse.

---

6. Summary

Phase	Focus	Builds on
1	Runner + core tool call tests	Existing test infra
2	All v1 scenarios + parameter matrix	Phase 1 runner
3	Crash recovery + durability	Phase 2 scenarios
4	Functional + multi-agent	Phase 3 durability
5	LLM judge + observability + CI	Everything above

Each phase is self-contained and delivers value independently. Phase 1 can start stabilizing changes immediately. Later phases expand coverage without requiring rework of earlier phases.